Fibonicci Series

mention how we will just solve question as we move forward & also how things will be clear (not in YT)

1	1	2	3	5	8	13
0	1	2	3	4	5	6



Sum of numbers is equal to previous two numbers.

PMI, extended form (don't introduce as extended but explain ~~after~~ before)

Base Case :- Same f(0) or f(1)

IH : f(i) is true for all $i \leq R$

explain why it will work.

IS : Use (2) to prove f(k+1) is true

The above makes our framework a lot stronger as we can assume for everything less than n.

write code for fibonicci number.

explain with first example of single base case.

order and everything ← make sure to properly explain recursion tree ⟸ dry run as well

Make sure to run the dry code & tree after writing the code.

We are doing math & not recursion.

Q: Write a program for a given number n.

1] To print 1 to N.
2] To print N to 1.

mention on how to build a thought process.

make sure to explain the difference properly.

Also explain head vs tail recursion.

Assignment

Q-1 Sum of digits of a number

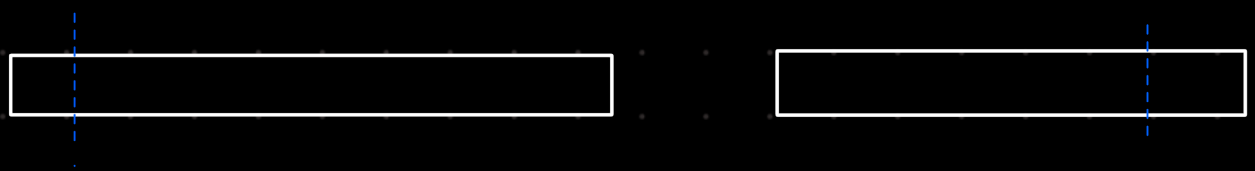
Q-2 Power of a number

Recursion with Arrays

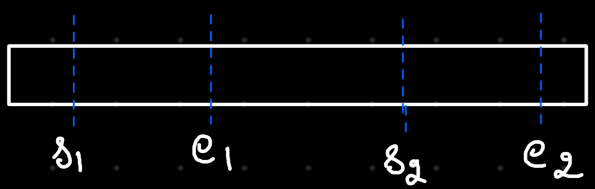
Whenever in arrays, we are dealing with a problem that can be broken & individually be applied to left over part, we can use recursion there.

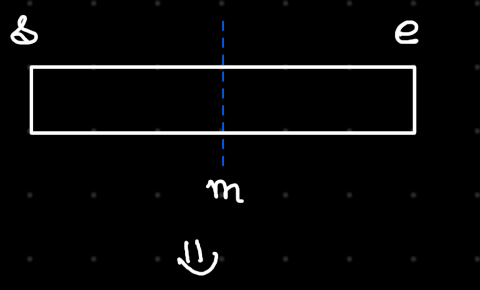
Below are major ways we break our array / list.

1. First or last element removed



2. Using start and end index (eg. binary search)





divide from middle

3. Copy part into a new array and send to further calls.

Q:- Sum of an array using Recursion

Find the sum of an array using recursion, specify the recurrence relation and recursion tree.

Give code solution via head and tail recursion.

Q: Check if array is sorted.

mention how these small problem gonna help in understanding.

using recursion.

explain how we handle array problem.

Now, we just have to think on our approach

arr[0] arr[1]

how to handle & write code better.

Now, in this explain the two approaches.

Keep on mentioning how understanding should also be clear.

↓

thought process

So it normally doesn't matter if we are doing our work first or not in these kind of problems.

1] Base Case

2] Recursive call

3] Our Calculation.

Q: Given an array find the first, last and all indices of a number.

We have to show multiple views here.

1] First Index
2] Last index
3] All index and passing the list
4] All index but list not getting passed

Explain using id

Can be in software dev practice so explain why we might want to send the list & update same only.

Also make sure to explain doubt which can come.

5] Point → not return nair but rather just call the function.

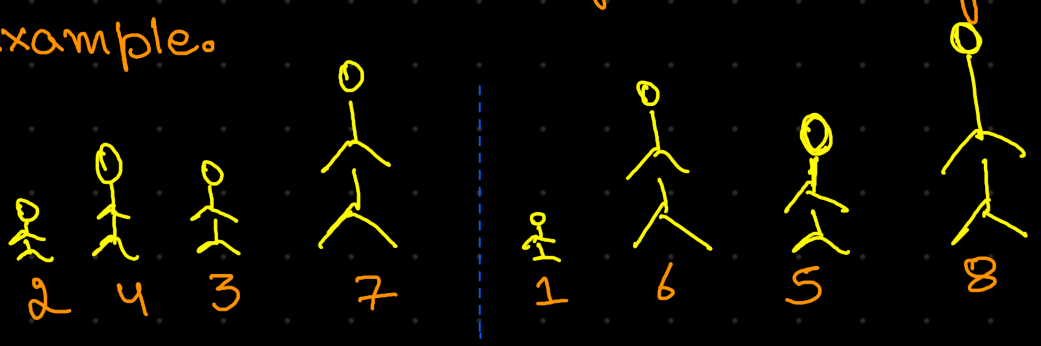
Also brush to create all the diagram and Python Tutor.

Merge Sort

⇒ recursive sorting algorithm

⇒ fast as compared to other sorting (will learn why)

Let's understand merge sort using an example.





first explain intuitively

use hackerank visualizer

list copy as in-place

how will it keep on going still?
even if left over part is sorted.

Quick Sort

⇒ In quick sort, unlike merge sort:

we will first do our work and then let recursion handle the rest.



---

we are now going to portion along last element.

Searching using recursion

→ Linear Search

tell how we can choose either the first element or the last element.

→ Binary search

Recursion with Strings

→ Just like arrays, for string too we work on them after piching one of the element and then asking recursion to work on rest.

→ we will be understanding the same via questions.

Replace Z from a string

```
def removeZ(string):
    if(string==""):
        return string

    if(string[0] in 'zZ'):
        return removeZ(string[1:])
    return string[0] + removeZ(string[1:])

removeZ("ZzA")
```

Check Palindrome

explain question and give example.

First explain the question and easy way via reversing.

then easy test case: nitin

difficult test case: 'was it a car or a a cat I saw

or

malay alam

```python
def is_palindrome_helper(s,start,end):
    if(start>=end):
        return True

    return s[start]==s[end] and is_palindrome_helper(s,start+1,end-1)

def is_palindrome(s: str) -> bool:
    if(len(s) == 0 or len(s)==1):
        return True


    # s = s.lower()
    s = "".join(c for c in s if c.isalnum()).lower()


    return is_palindrome_helper(s,0,len(s)-1)
```

Medium to difficult level recursion Question.

First explain importance of approach and intuition and making the thought process instead of chasing solution.

Also on focusing on pattern of these questions as they are asked in Tier 1 companies.

String, substring and subsequences

Example of buffet

String: a b c

Substring: (same order and continous)
a b c ab bc ~~ac~~
abc
$\Rightarrow \frac{n(n+1)}{2}$

Subsequences (in order but don't need to be continous)
a b c
ab bc ac
abc
2^n

Array: [1,2,3]

Subarray == substring

Subset $\Leftrightarrow$ without order and sequence but duplicate only once.

Subsequences of a string (Print, return, list)

First now again pattern and answer

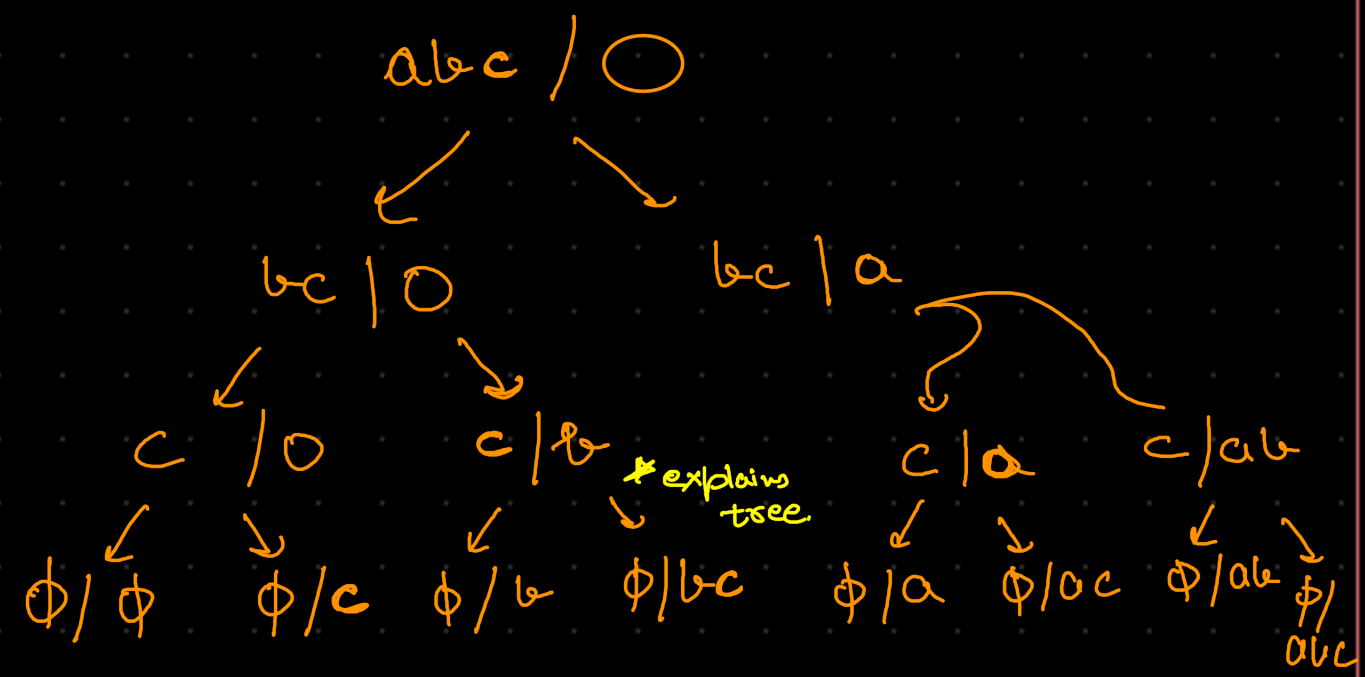
taking and not-taking ⇒ subsequence

Print all subsequences.

explain the intuition with your budget example

start from base case then build it up.

{friend & you}



Use above tree to properly explain & also on where to add input[0].

Explain all the approaches of print and return

tell how passing in an argument is easy and how we have to create and return

diff. approaches.

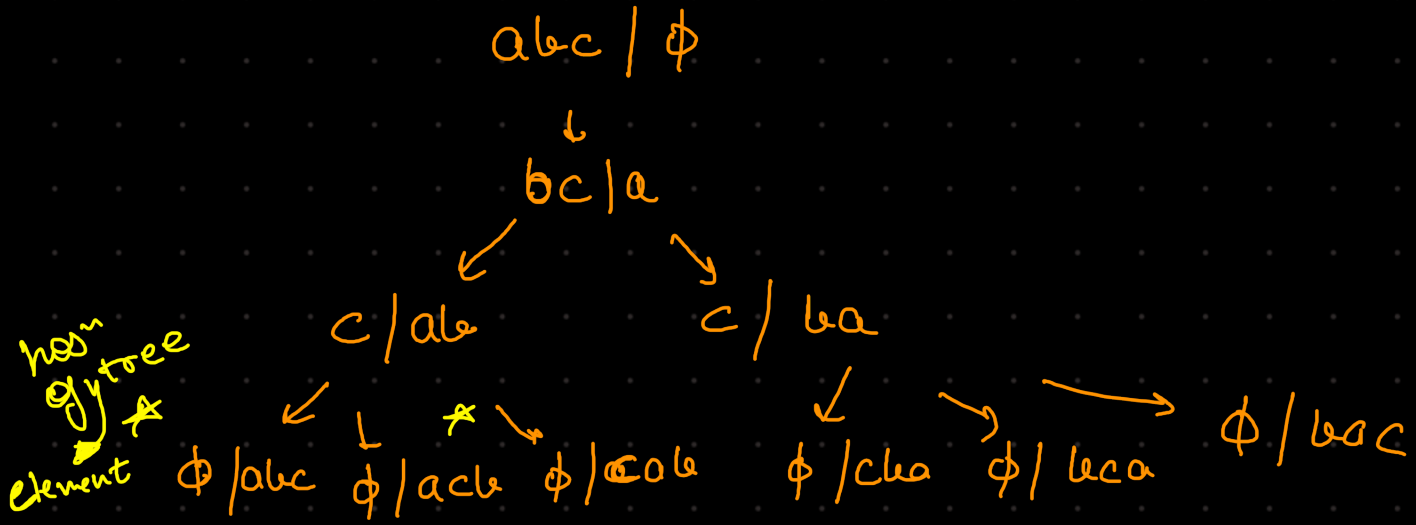
ask to watch previous video for sure.

Permutations

Subset : take 1 and remove one before, not now

talk in term of your approach.

recursion breaking problem into smaller problem



Explain how f^n calls are increasing.

dependent on output

variable f^n calls ⇒ for loop

so mention all these and also how easy but everything after code.

Make fallback on your video.

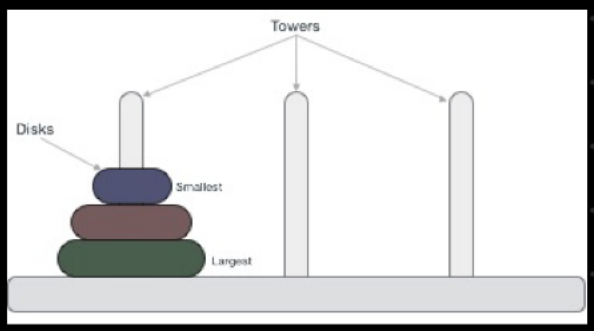
Make sure to cover every approach & specifically mention how

i) useful in DP
ii) FAANG / tier-1 interview.
iii) understanding and writing code.

Print and return all combination

Make sure to mention how it is asked in FAANG companies

Tower of Hanoi



Letter Combination of a Keypad

First explain the problem properly.

with images and example.

2 3

Make tree as well
and explain
intuitively

Print and return all codes

Framework to solve any recursion question.

1] Identify if problem can be broken into a smaller same problem.

2] Write recurrence relation.

3] Code

a) Identify and write base cases. [can be more than 1]

b) Assume (don't Question) the Induction hypothesis/ get the small answer



c] Do your work and make sure to consider all the test cases and edge cases

→ Draw recursion tree or go in depth after code writing.

If we have to print or share some information across recursive call.

use function parameter.

Make sure to crack the order or sequence by taking a small example.